Digital VLSI Design Assignment

4-bit Array Multiplier
RTL to GDSII

Open-Source ASIC Design Flow using SKY130

Team Members

Animesh Pandey	Roll No. 2026203012
Polu Lohith Reddy	Roll No. 2026203004

Technology:	SkyWater SKY130, `sky130_fd_sc_hd`
RTL / verification:	Verilog, Icarus Verilog, GTKWave
Synthesis / timing:	Yosys, OpenSTA
Physical design:	LibreLane, OpenROAD, Magic, KLayout, Netgen
Repository:	`github.com/Animeshh-22/triode-repository`

October 26, 2026

Abstract

This report documents the implementation of a 4-bit unsigned array multiplier and its complete open-source RTL-to-GDSII flow. The Verilog design forms 16 partial products, uses 12 full-adder instances, registers the two input operands and the 8-bit product, and provides a two-clock-cycle input-to-output latency. Exhaustive RTL and functional gate-level simulations check all 256 operand pairs. The design is mapped to the SKY130 high-density standard-cell library, analyzed with OpenSTA, and physically implemented with LibreLane to produce the final GDSII layout.

1 Introduction and Design Background

The objective is to design and verify a 4-bit by 4-bit unsigned array multiplier, synthesize it to the SKY130 standard-cell library, evaluate its timing, and complete place and route through an open-source toolchain. For inputs $a, b \in [0, 15]$, the result $p = a \times b$ ranges from 0 to 225 and therefore requires eight bits. The submitted design uses an asynchronous active-low reset, registers both input operands, and registers the product at the output.

Table 1: Design specification

Parameter	Specification
Operands and result	Two 4-bit unsigned inputs, a and b; 8-bit registered output p
Array structure	16 partial-product AND operations and 12 full adders
Registers	8 input-register bits and 8 output-register bits; 16 flip-flops total
Reset	Asynchronous, active low (`rst_n`)
Latency	Two rising clock edges from sampling the operands to registering the product
Technology	SKY130 130 nm process, `sky130_fd_sc_hd` standard-cell library

1.1 Shift-and-add multiplication

For each bit b_i of the multiplier, a row of partial products is formed by ANDing b_i with every bit of a. The row is shifted by i bit positions and the four rows are added. For an N-bit array multiplier, the number of partial-product operations scales as N^2, while the standard array structure uses $N(N-1)$ full adders. Its longest combinational path grows approximately with $2N$ full-adder delays. These relationships provide a useful architectural estimate to compare with synthesis and static timing results.

A full adder combines three one-bit inputs: the partial-product bit, the bit passed from the preceding row, and the carry from the previous stage. Its Boolean equations are

$$s = a \oplus b \oplus c_{in}, \qquad c_{out} = (ab) \vee (ac_{in}) \vee (bc_{in}).$$

The sum bit stays in the current product column, while the carry contributes to the next column. The same pattern repeats across each row, making the array structurally regular and straightforward to describe and verify in RTL.

The input and output registers create a two-stage pipeline: an input pair is first sampled into a_{reg} and b_{reg}, and the corresponding combinational product is captured by p on the following rising edge. The registered interface makes the latency explicit and allows Part C timing analysis to examine the register-to-register combinational path.

2 RTL Design and Verification (Part A)

2.1 Architecture and datapath

The module `mult_array` first samples a and b into two 4-bit registers on each rising edge of `clk`. The registered operands feed the combinational array. The array's 8-bit result is captured in the output register `p`. The input registers and output register are all asynchronously cleared when `rst_n`=0. There are eight input flip-flops and eight output flip-flops, giving 16 flip-flops in total.

The array is built as four partial-product rows. Row 0 contains only four AND operations; its least-significant bit is $p[0]$, and the other three bits pass to the next row. Rows 1, 2 and 3 each contain four AND operations and a four-stage ripple-carry chain. The least-significant sum bit of each of these rows becomes $p[1]$, $p[2]$ and $p[3]$ respectively. The remaining sum bits and the final carry are passed down, and the final row generates $p[7:4]$. Each full-adder row begins with a carry-in of zero.

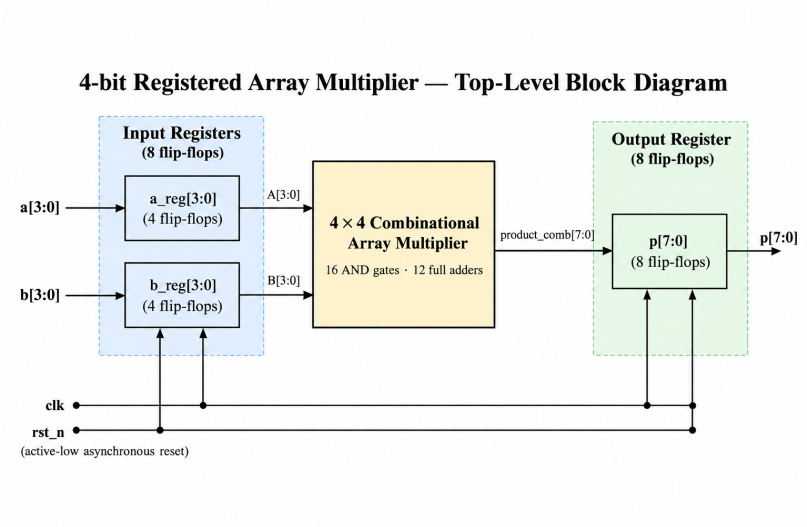


Figure 1: Top-level organization: input registers, combinational array, output register, clock and reset.

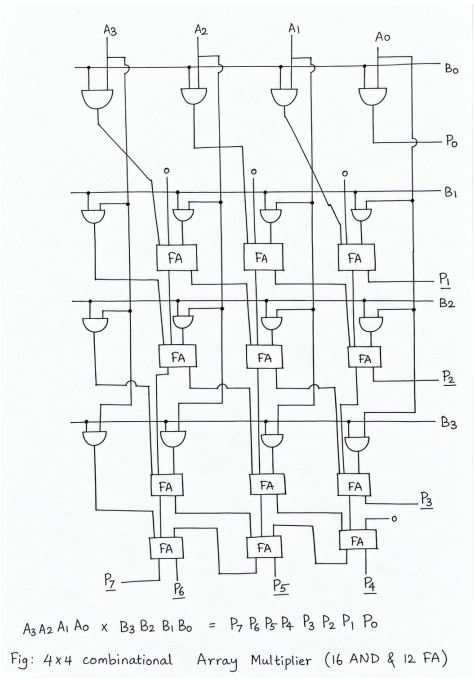


Figure 2: Array datapath showing the partial-product rows, 12 full adders, and product bits.

For partial-product row i, the bit-level expression is $pp_i[j] = a[j] \wedge b[i]$. The rows are shifted by their row index before addition. The RTL instantiates 12 explicit `full_adder` modules and does not use the multiplication operator in the design module; the multiplication operator is used only by the testbench to calculate expected values.

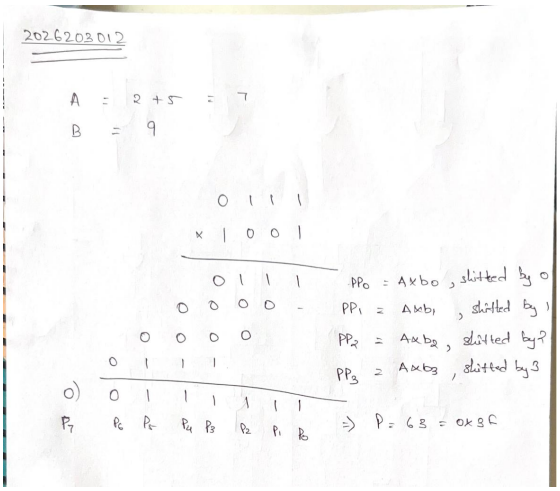
2.2 Hand-worked example

The assignment's example is derived from the last digit of roll number 2026203012: $A = 2 + 5 = 7$. Because $A = 7$, the required second operand is $B = 9$. In binary, $A = 0111_2$ and $B = 1001_2$. Only b_0 and b_3 are one, so two partial-product rows contribute to the result.

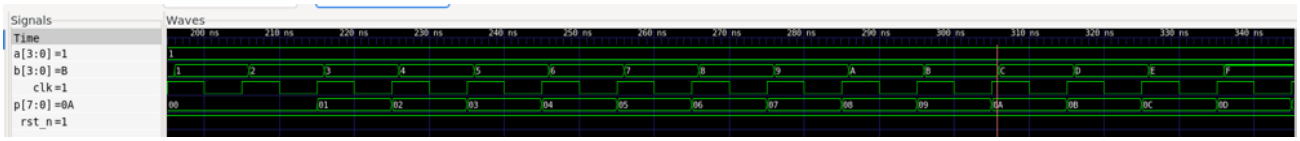
Table 2: Shifted partial products for 7×9

Row	8-bit shifted value
$pp_0 = (0111 \wedge b_0)$, shift 0	00000111
$pp_1 = (0111 \wedge b_1)$, shift 1	00000000
$pp_2 = (0111 \wedge b_2)$, shift 2	00000000
$pp_3 = (0111 \wedge b_3)$, shift 3	00111000
Sum	00111111

Therefore, $7 \times 9 = 63 = 00111111_2 = \texttt{0x3F}$. The handwritten arithmetic and the corresponding waveform are included as evidence that the worked example matches the simulated design.



(a) Hand-worked binary multiplication for $a = 7$ and $b = 9$.



(b) GTKWave evidence for $p = 0x3F$, showing the two-cycle latency.

Figure 3: Hand-worked arithmetic example and matching simulation waveform.

2.3 Testbench and verification result

The testbench uses a 10 ns clock, holds reset low for the first two rising edges, drives all 256 input combinations, and compares the result after respecting the two-cycle latency. The passing RTL run reports `PASS: 256 vectors checked, 0 errors`. To validate that the testbench detects an incorrect adder, the sum equation was temporarily changed to $a \oplus b$ (omitting the carry-in). The diagnostic run reported `FAIL: 256 vectors checked, 95 errors`. This confirms that the self-checking testbench is sensitive to an arithmetic fault; the correct full-adder equation is used in the final RTL.

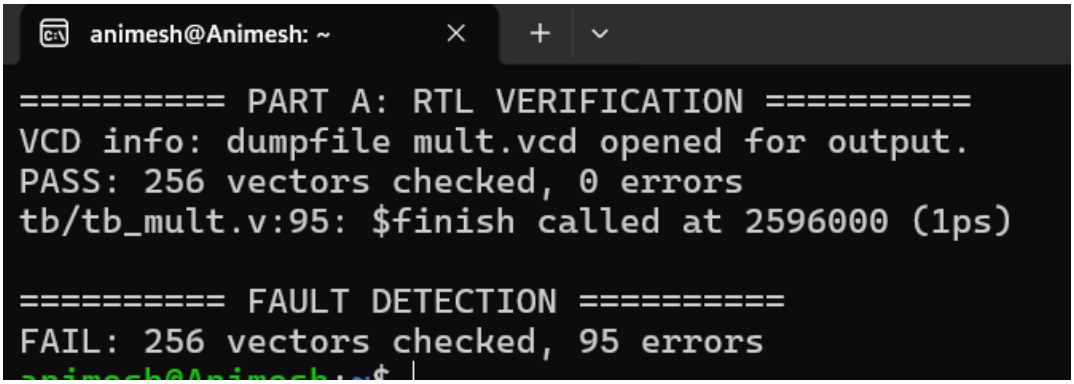


Figure 4: Terminal evidence for the exhaustive RTL pass and the fault-detection check.

3 Logic Synthesis with Yosys (Part B)

Yosys converts the RTL hierarchy into a technology-mapped netlist. The synthesis script reads `rtl/full_adder.v` and `rtl/mult_array.v`, runs generic synthesis with the `mult_array` top module and flattening, maps the flip-flops through `dfflibmap`, maps combinational logic through ABC using the SKY130 Liberty library, removes unused logic, writes the statistics report, and exports the gate-level Verilog netlist.

The completed outputs are `synth/synth.tcl`, `synth/mult_array_netlist.v` and `synth/mult_array_stat.rpt`. The netlist is then simulated using the SKY130 functional cell models and the same exhaustive self-checking testbench. The functional gate-level run reports `PASS: 256 vectors checked, 0 errors`.

Table 3: Part B synthesis results from `mult_array_stat.rpt`

Metric	Measured value
Total mapped standard cells	63
Flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Combinational cells (total minus flip-flops)	47
Total mapped cell area	740.7104 μm^2
Flip-flop area	400.384 μm^2
Flip-flop area as a share of total	54.05%
Three most-used cell types	`dfrtp_1`: 16; `nand2_1`: 8; `a21oi_1`: 6
Functional gate-level simulation	`PASS: 256 vectors checked, 0 errors`

Table 4: Main synthesis steps and their purpose

Yosys step	Purpose
`read_verilog`	Reads the full-adder and multiplier RTL modules.
`synth -top mult_array -flatten`	Elaborates the design and flattens the full-adder hierarchy for optimization.
`dfflibmap -liberty`	Maps sequential elements to compatible library flip-flops.
`abc -liberty`	Maps and optimizes combinational logic using SKY130 library cells.
`opt_clean -purge`	Removes unused wires and cells after mapping.
`stat` and `write_verilog`	Saves the cell/area statistics and writes the mapped netlist.

The 63 mapped cells are not a direct sum of 16 AND gates and 12 full-adder blocks. After flattening, ABC can combine Boolean terms, change signal polarity and select compound library cells. The number of mapped cells therefore describes the optimized standard-cell implementation rather than a one-to-one count of the RTL primitives. The flip-flops use 400.384 μm^2 of the 740.7104 μm^2 mapped area, or 54.05%, because all operand bits and all result bits are registered.

```
========== PART B: YOSYS SYNTHESIS ==========
       63   740.71 cells
   Chip area for module '\mult_array': 740.710400
     of which used for sequential elements: 400.384000 (54.05%)

========== GATE-LEVEL FUNCTIONAL SIMULATION ==========
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:95: $finish called at 2596000 (1ps)

========== UNIT-DELAY DIAGNOSTIC ==========
FAIL: 256 vectors checked, 239 errors
```

Figure 5: Terminal evidence for the Yosys statistics and the passing functional gate-level simulation.

4 Static Timing Analysis with OpenSTA (Part C)

4.1 Method and minimum-period estimate

OpenSTA evaluates timing paths using the typical SKY130 HD corner (`tt_025C_1v80`). The script reads the Liberty timing library, loads and links `synth/mult_array_netlist.v`, creates a clock on `clk`, applies 1.0 ns input and output delays, reports the worst setup slack, and writes the maximum-delay path into `sta/mult_array_timing.rpt`.

The baseline clock period is 10 ns. The measured worst setup slack is 7.396 ns, so the arithmetic minimum-period estimate is

$$T_{\min} = T_{\text{baseline}} - WNS = 10 - 7.396 = 2.604 \text{ ns}.$$

Rounding upward to the next 0.01 ns gives a verification target of $T = 2.61$ ns. The detailed critical-path report at this period records +0.006 ns setup slack, which is close to zero as expected for a minimum-period check. The reset-recovery check has +1.838 ns slack.

Table 5: Part C timing results

Metric	Result
Worst setup slack at $T = 10$ ns	7.396 ns
Arithmetic minimum-period estimate	2.604 ns
Rounded verification period	2.61 ns
Setup slack at $T = 2.61$ ns	+0.006 ns (MET)
Reset-recovery slack at $T = 2.61$ ns	+1.838 ns (MET)
$F_{\max}$ from unrounded $T_{\min}$	$1000/2.604 \approx 384.0$ MHz
Frequency at rounded 2.61 ns	$1000/2.61 \approx 383.1$ MHz
Two-cycle latency estimate	$2T_{\min} \approx 5.21$ ns
Throughput estimate	Approximately 384.0 million results/s
Critical path	`_095_` to `_092_`
Launch / capture registers	$a_reg[1]$ to $p[6]$
Logic cells between registers	8

The frequency figure based on unrounded $T_{\min}$ is used for the assignment's $F_{\max} = 1000/T_{\min}$ estimate. The rounded 2.61 ns clock is the separately measured verification point; its corresponding period-based frequency is 383.1 MHz.

```
animesh@Animesh:~$ docker exec -it iic-osic-tools_xvnc_uid_1000 bash -lc '
set -o pipefail
cd /foss/designs/multU_2026203012

echo "=== OPENSTA BASELINE: T = 10 ns ==="
sta -no_splash -exit sta/sta.tcl 2>&1 | grep -i "worst slack"

echo
echo "=== OPENSTA TARGET: T = 2.61 ns ==="
T=2.61 sta -no_splash -exit sta/sta.tcl 2>&1 | grep -i "worst slack"

echo
echo "=== CRITICAL-PATH REPORT ==="
grep -E "Startpoint:|Endpoint:|slack \\(MET\\)" sta/mult_array_timing.rpt
'
[INFO] Final PATH variable: /headless/.local/bin:/foss/tools/bin:/foss/tools/sak
oss/tools/kactus2:/foss/tools/klayout:/foss/tools/osic-multitool
[INFO] Final PYTHONPATH variable: /headless/.local/lib/python3.12/site-packages:
ib-dynload:/usr/local/lib/python3.12/dist-packages:/usr/lib/python3/dist-package
ools/openems/local/lib/python3.12/dist-packages:/foss/tools/pyopus/local/lib/pyt
s/tools/klayout/pymod:/foss/tools/vacask/lib/vacask/python
=== OPENSTA BASELINE: T = 10 ns ===
worst slack max 7.40

=== OPENSTA TARGET: T = 2.61 ns ===
worst slack max 0.01

=== CRITICAL-PATH REPORT ===
Startpoint: rst_n (input port clocked by clk)
Endpoint: _086_ (recovery check against rising-edge clock clk)
          1.838   slack (MET)
Startpoint: _095_ (rising edge-triggered flip-flop clocked by clk)
Endpoint: _092_ (rising edge-triggered flip-flop clocked by clk)
          0.006   slack (MET)
animesh@Animesh:~$
```

Figure 6: Terminal evidence for the 10 ns baseline, the 2.61 ns verification run and the reported setup slack.

4.2 Critical-path cell sequence

The saved report identifies a register-to-register maximum-delay path from mapped flip-flop `_095_` (corresponding to $a_reg[1]$ in the netlist) to mapped flip-flop `_092_` (corresponding to $p[6]$). Eight mapped logic cells lie between the launch and capture flip-flops. Their incremental delays and cumulative arrivals are listed below.

Table 6: Critical-path cells at $T = 2.61$ ns

Stage	Cell type	Delay (ns)	Arrival (ns)
Launch	`dfrtp_1 (clk-to-Q)`	0.398	0.398
1	`clkinv_1`	0.074	0.472
2	`o311ai_0`	0.348	0.820
3	`a21oi_1`	0.172	0.992
4	`maj3_1`	0.381	1.373
5	`xnor2_1`	0.160	1.534
6	`maj3_1`	0.385	1.918
7	`maj3_1`	0.377	2.295
8	`xnor3_1`	0.175	2.471
Capture	`dfrtp_1 (setup 0.134 ns)`	–	2.471

Three majority cells (`maj3_1`) implement carry-like Boolean functions, while the XNOR cells contribute to sum logic. Because the RTL was flattened and the logic remapped by synthesis, standard-cell boundaries do not preserve a one-to-one mapping to the twelve source-level full-adder instances. At the architectural level, the path length is comparable with the approximate $2N = 8$ full-adder-delay estimate for a 4-bit array. The cell-by-cell table provides the implementation-level view used for the timing calculation.

For the measured path, the data arrival time is approximately 2.471 ns. The required time for the 2.61 ns clock is the clock period less the capture flip-flop setup time: $2.610 - 0.134 = 2.476$ ns. Thus the report's unrounded internal values give a small positive setup margin of approximately 0.006 ns. This is consistent with using the rounded period as a verification point near the arithmetic minimum-period estimate.

5 Physical Design: RTL to GDSII (Part D)

5.1 Flow and selected configuration

LibreLane automates synthesis, floorplanning, placement, clock-tree synthesis (CTS), global and detailed routing, parasitic extraction, timing analysis and physical verification. The final configuration is `pnr/config.json`, with `DESIGN_NAME = mult_array`, SKY130 HD standard cells, absolute floorplan sizing, a fixed 150 μm $\times$ 150 μm die, and `CLOCK_PERIOD = 7.5` ns. The final metrics and layout were copied to `pnr/metrics.json` and `pnr/mult_array.gds`.

The physical-design stages have distinct purposes. Floorplanning fixes the die/core geometry and establishes the power grid. Placement assigns standard cells to rows. CTS distributes the clock to the register bank. Routing connects the cells with metal interconnect. Extraction and signoff evaluate parasitic effects and verify the layout. The resulting GDSII is the layout database delivered for the selected implementation.

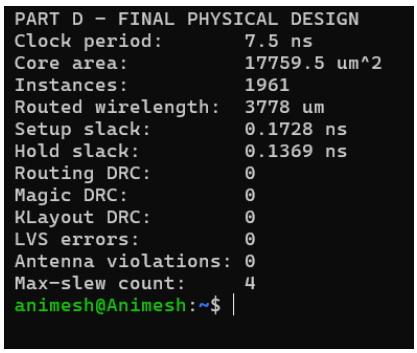


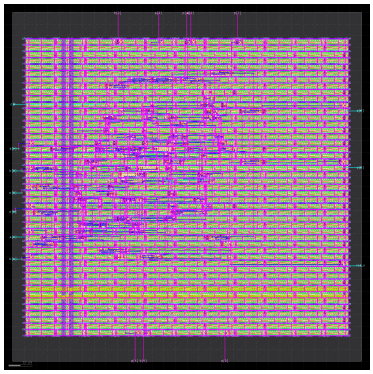
Figure 7: Terminal evidence for the selected 7.5 ns implementation and its saved physical-design metrics.

5.2 Measured results and final layout

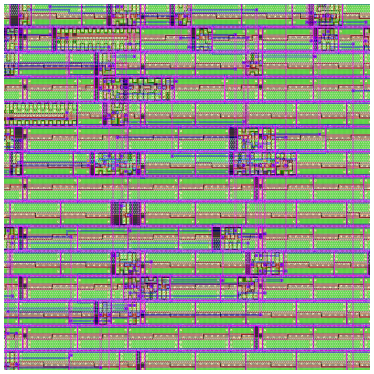
Table 7: Part D results for the selected final implementation

Metric	Result
Clock period	7.5 ns
Recorded LibreLane runs	24
Die area	$22,500\ \mu m^2$ ($150\ \mu m \times 150\ \mu m$)
Core area	$17,759.5\ \mu m^2$
Part B synthesis cell count	63
Place-and-route instances	1,961 total: 386 standard-cell instances and 1,575 fill cells
Non-physical cells	141 (logic, flip-flops and buffers)
Cell categories added	245 tap cells; 1,575 fill cells; 27 timing-repair buffers; 3 clock buffers; 0 decap cells
Routed wirelength / vias	3,778 μm / 967 vias
Worst-corner setup slack	+0.1728 ns
Typical-corner setup slack	+3.7137 ns
Worst-corner hold slack	+0.1369 ns
Post-route F_{max}, worst corner	$1000/(7.5 - 0.1728) \approx 136.5$ MHz
Post-route F_{max}, typical corner	$1000/(7.5 - 3.7137) \approx 264.1$ MHz
Part C F_{max} estimate	Approximately 384.0 MHz
DRC errors (routing, Magic and KLayout)	0
LVS errors / antenna violations	0 / 0

For comparison, the mapped Part B design contains 63 cells, while the physical-design database contains 1,961 instances because the flow adds the standard cells needed for layout implementation, well taps, fill, clock distribution and timing repair. The 386 standard-cell instances include the functional and physical standard cells; the additional 1,575 fill instances occupy otherwise unused row space and support layout density requirements. The selected run has positive setup and hold slack and reports zero DRC, LVS and antenna errors.



(a) Whole-die view.



(b) Zoomed standard-cell rows and metal wires.

Figure 8: Layout screenshots rendered from the final GDSII in KLayout.

6 Answers to the Assignment Questions

1. **How many logic cells did synthesis produce, and why is the count not $16 + 12$?** The mapped netlist contains 63 cells in total, including 16 flip-flops, so 47 cells are combinational. The 16 partial-product operations and 12 full-adder instances are RTL-level structures; synthesis flattens and optimizes their Boolean equations, merges logic where possible, and maps the result to available SKY130 cells. Compound gates and polarity optimization mean the source-level gate/adder count does not translate one-to-one into library cells.

2. **What is the critical path, how many cells does it contain, and how does that compare with $2N$?** OpenSTA reports the maximum-delay register-to-register path from $a_reg[1]$ (mapped cell `_095_`) to $p[6]$ (mapped cell `_092_`). Eight mapped logic cells lie between the launch and capture flip-flops. This is consistent in scale with the approximate $2N = 8$ full-adder-delay estimate for $N = 4$. The mapped cells are optimized standard-cell functions, so they should not be interpreted as eight literal full-adder blocks.

3. **What percentage of area is occupied by flip-flops, and why is it substantial?** The 16 mapped flip-flops occupy 400.384 μm^2 out of 740.7104 μm^2. Their share is $400.384/740.7104 \times 100 \approx 54.05\%$. This is substantial for a small datapath because the design registers all eight operand bits and the full eight-bit product, while the combinational arithmetic is compact after technology mapping.

4. **Why is post-route $F_{\max}$ lower than Part C, and what cells are added?** Part C estimates timing on the mapped netlist with an ideal clock network and no routed interconnect parasitics. Physical design includes clock distribution, actual wire resistance and capacitance, placement effects and timing repair. The selected layout has $F_{\max} \approx 136.5$ MHz at the worst setup corner and 264.1 MHz at the typical corner, compared with approximately 384.0 MHz from Part C. Tap cells connect well regions to supply rails and help prevent latch-up; fill cells meet row and density requirements; timing-repair buffers improve electrical drive and timing; and clock buffers distribute the clock to sequential cells.

5. **What do DRC and LVS verify, and why are both required?** Design-rule checking (DRC) checks geometric manufacturing constraints such as minimum width, spacing, enclosure and related layout rules. Layout-versus-schematic (LVS) extracts the layout connectivity and compares it with the intended circuit netlist. Both are required because geometric compliance alone does not prove the circuit is connected correctly, while a matching netlist alone does not prove the geometry follows manufacturing rules. The selected run reports zero DRC and zero LVS errors.

6. **How would the design scale to an 8-bit multiplier? Show the estimate.** For an N-bit array, the partial-product count is N^2 and the full-adder count is $N(N-1)$. Thus, an 8-bit array requires $8^2 = 64$ partial-product AND operations and $8(8-1) = 56$ full adders, compared with 16 and 12 for the 4-bit design. The architectural critical-path estimate increases from roughly $2N = 8$ to $2N = 16$ full-adder delays. Using the measured eight-cell logic delay of approximately 2.072 ns, a first-order estimate is $T_{\min,8} \approx 0.398 + 2(2.072) + 0.134 = 4.676$ ns, so $F_{\max,8} \approx 1000/4.676 \approx 214$ MHz. This is a scaling estimate, not a measurement of an 8-bit implementation.

7 Individual Contributions

Team member	Roll number	Contribution
Animesh Pandey	2026203012	Led the implementation and integration across Parts A–D, including RTL design and exhaustive verification, synthesis and gate-level validation, OpenSTA timing and critical-path analysis, LibreLane physical-design execution, collection of final metrics and layout evidence, repository organization, and preparation of the report.
Polu Lohith Reddy	2026203004	Reviewed the Part B synthesis statistics and functional gate-level simulation results, and contributed a focused review of the Part C timing table.

8 Conclusion

The 4-bit unsigned array multiplier was implemented with registered inputs, a combinational array of partial-product and adder logic, and a registered 8-bit output. Exhaustive RTL simulation and functional gate-level simulation both checked all 256 operand combinations successfully. Synthesis produced 63 mapped cells with a total area of 740.7104 μm^2. OpenSTA estimated $T_{\min} = 2.604$ ns and measured +0.006 ns setup slack at the rounded 2.61 ns verification period. The selected 7.5 ns LibreLane implementation produced the final GDSII layout with positive setup and hold slack and zero DRC and LVS errors.